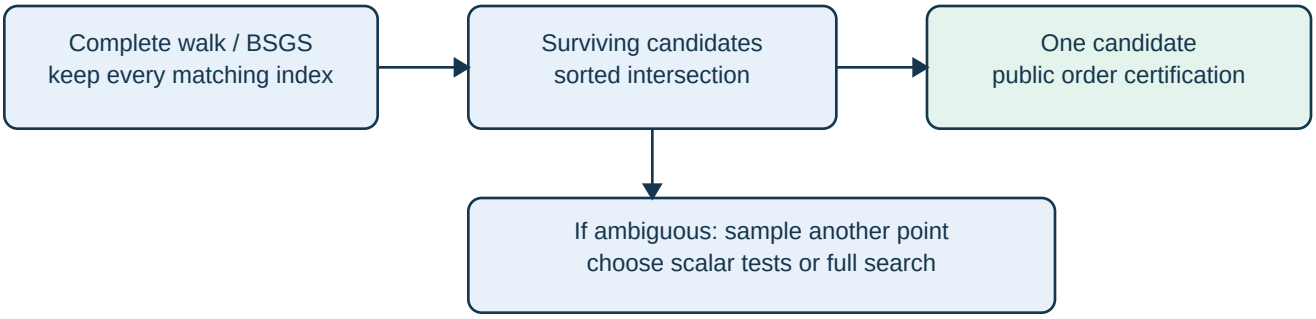
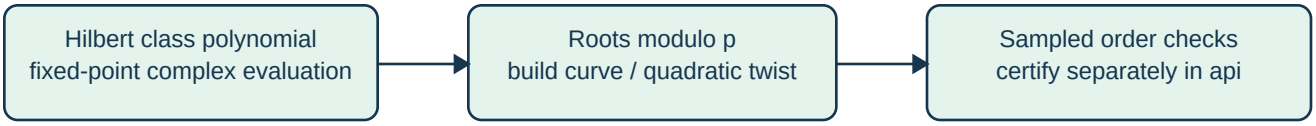
SEA, point counting, and CM generation

Implemented point-count changes



Existing CM generation (specspecialized research constructor)



Date: 2026-10-15. Baseline: aburan28/crypto at cf2745214d83e7bbf737f07d758e486d6af8fa7f. Scope: the standalone isogeny_algos crate. This is an implementation review and correctness-validation record. It contains no measured performance comparison or new mathematical theorem.

Findings and changes

Candidate completeness. The original SEA baby table stored one index per point (`baby.entry(cur).or_insert(jj)`). When the step point has small order, different indices represent the same point. Keeping one loses possible traces before Atkin filtering and intersection with later samples. The new helper retains every matching index, including repeated infinity, and handles a partial last giant block. Its table size uses the exact integer ceiling of the square root instead of a floating-point approximation.

The small-field counter had a related defect: repeated infinity was recorded with index zero instead of its actual index. That is corrected too. Neither fix changes the independent public order-certification conditions.

Adaptive refinement. Previously each sampled point triggered another full walk or BSGS over the original interval. Both counters now choose scalar tests on the surviving candidates when a conservative point-addition estimate favors that work. They retain the full search when many candidates survive. This changes the follow-up work from a search across the original interval to testing the remaining orders in those cases. It does not assume that a typical first pass leaves multiple candidates or that scalar tests always win.

Sorted intersections. Search hits are ordered, so intersection now uses binary search instead of a linear membership scan for every prior candidate. For C previous candidates and H hits, the membership work becomes O(C log H) instead of O(C H). The small-field counter also computes its fixed m-times-point offset once per sampled point instead of once per giant step.

These are established algorithm and implementation changes. Their wall-clock effect, memory crossover, and end-to-end gain are unmeasured. Retaining repeated indices uses more storage than discarding them. The estimate used to choose scalar tests counts point additions; it does not model field inversions, hash allocation, or hardware-specific costs.

Further algorithmic improvements, proposed

First: use Atkin information during recombination. Currently Atkin sets only filter group-search hits. Build a bounded CRT sieve or meet-in-the-middle recombination over selected Atkin sets so they reduce the search itself. Choose sets by candidate-product size and information gained; retain all residues and an explicit memory cap. Compare against the current method on identical orders, including unfavorable Atkin distributions. No crossover has been measured here.

Second: reduce inversions in point arithmetic. The shared padd routine uses affine formulas and a field division for each ordinary addition. A Jacobian walk with batch conversion to canonical affine keys can reduce those inversions. Projective coordinates cannot be hashed as raw triples, because different triples may represent the same point. Verify infinity, doubling, inverse points, and batch normalization before comparing costs across field sizes.

Third: replace the linear Elkies eigenvalue scan. elkies_eigenvalue compares successive symbolic kernel points for up to half the auxiliary prime. A BSGS discrete-log search can reduce the number of quotient-ring point operations from linear to square-root growth in that prime. Handle noninvertible denominators and the y-coordinate sign explicitly; retain the existing Frobenius check. Extra table and normalization costs may outweigh the reduction for the small primes currently used.

For repeated counts: reuse modular polynomials. The public count_points entry point constructs a fresh Phi cache for each count. A field-bound counting context could reuse it across curves. This is amortized setup reuse, not an improvement to a single cold count. Cache identity must include the full field modulus and representation, not only its bit length or low word.

CM support is already present

The baseline already contains fixed-point complex-analytic Hilbert class polynomials in cm::hilbert_class_poly. gls_loop::find_instance searches for GLS parameters; gls_loop::build_curve reduces the polynomial modulo p, finds all roots, constructs a curve, and selects its quadratic twist using the requested order. The existing loop_closes_and_maps_points_on_a_small_instance test runs this construction at a 40-bit prime. It passed on the unmodified baseline.

This is specialized research support. The search imposes p = 3 modulo 4 and a small-cofactor GLS twist. The constructor requires complete splitting and rejects j = 0 and 1728. There is no generate-cm CLI command. Hilbert coefficient rounding checks are not certified interval arithmetic; eight sampled annihilation checks do not prove an exact order or a full endomorphism ring. The supported API separately certifies proposed orders with point-order divisors and Hasse uniqueness, subject to its documented probable-prime checks.

Because CM generation exists, this change does not introduce a second CM implementation. It documents the existing Rust workflow and adds a direct enumeration check of a generated degree-three Hilbert-polynomial instance. That finite regression verifies the requested trace independently of the constructor's sampled annihilation tests. It is not a general CM correctness proof or a certificate of the endomorphism ring.

Validation and execution limits

The baseline library command passed 20 tests with one pre-existing ignored exploratory test. A first candidate validation passed 21 library tests, seven API/CLI tests, and eight SEA tests, with the same ignored library test. The final run passed 22 library, seven API/CLI, three kernel, and eight SEA tests: 40 passed, with one pre-existing ignored library test. It includes the independent CM regression. The log is ../validation.txt. Clippy with warnings denied and formatting checks passed. No ignored test was enabled or disabled.

The new BSGS regression exhaustively compares every base and step point on three small-field fixtures, including infinity, at seven interval lengths including 1025. An independent square-multiplicity reference checks complete SEA recombination with no odd auxiliary primes over three field sizes, three models, and four RNG seeds per model. Existing tests compare SEA with BSGS at 40 and 61 bits, check sampled annihilation at 127 bits, and cover Atkin towers, Elkies cycles, standard-curve certificates, and CLI status records. The 127-bit annihilation test is a consistency check, not an independent count.

Host: Linux x86_64, AMD EPYC 9V74, five visible logical CPUs. Toolchain: rustc 1.99.1, stable x86_64-unknown-linux-gnu. These were correctness runs with no benchmark isolation or measured speedup. No benchmark figures, curve registry entries, ECDLP claims, or canonical performance graphs were promoted. The existing cost table in the usage guide remains historical and unchanged.

The initial shell clone failed because network permission was not enabled for that command; the enabled network path succeeded. Rust was installed in an isolated workspace toolchain. conductor check exited 127 because Conductor is not installed. The repository's docs/CONDUCTOR.md documents the unavailable CLI and absent .conductor/ scaffolding; no shared reservation was acquired. Changes were made in a fresh isolated feature branch. This coordination gap is an environment limitation, not an algorithmic obstruction.

Sources and reproduction

- Baseline source: https://github.com/aburan28/crypto/tree/cf2745214d83e7bbf737f07d758e486d6af8fa7f/isogeny_algos

- SEA recombination and eigenvalues: ../../../isogeny_algos/src/find/sea.rs

- Affine arithmetic and small-field counter: ../../../isogeny_algos/src/curve.rs

- Field dispatch, count dispatch, and certification: ../../../isogeny_algos/src/api.rs

- Hilbert polynomial computation: ../../../isogeny_algos/src/cm.rs

- Existing CM constructor and independent regression: ../../../isogeny_algos/src/gls_loop.rs

- Independent count checks and existing SEA suite: ../../../isogeny_algos/tests/sea.rs

- API/CLI checks: ../../../isogeny_algos/tests/api.rs

- Workflow and validation commands: ../README.md

- Diagram source and PDF renderer: render.py; vector output: flow.svg

The diagram describes implemented control flow and existing CM support. It has no quantitative axes because no comparable timing observations were collected.